

BME 5650 – HW #4, Task 3

MLP for Joint Reaction Force: Varying the Learning Rule and Activation Function

Submission: rg.mcghee2@gmail.com | Notebook: HW4_Task3_MLP_LearningRate_Activation.ipynb

1. Assignment and Approach

Task 3 asks: “Run the MLP with varying values for the learning rule and activation function.” The course-default configuration provided in the in-class notebook (`jrf_ann2.ipynb`) is: 80% / 20% train/validation split, one hidden layer of 8 neurons, ReLU activation, and a learning rate of 0.01. Starting from that notebook, the hidden-layer activation function and the Adam optimizer's learning rate (the “learning rule”) were converted from hard-coded values into parameters of `build_model()`, and the model was retrained across a full 3×3 grid of:

- **Activation function:** relu (course default), tanh, sigmoid
- **Learning rate:** 0.001, 0.01 (course default), 0.1

Everything else — the data split, feature/target standardization, hidden-layer width, L2 regularization ($1e-2$), EarlyStopping policy (patience = 40, restore best weights), 500-epoch cap, and random seed (4) — was left unchanged so that the effect of activation and learning rate could be isolated.

2. What Was Changed in the Code

2.1 Parameterized model builder

The original `build_model()` hard-coded `activation="relu"` and `learning_rate=0.01` inside the function body. It was changed to accept both as arguments (with the course defaults kept as the function's default values, so calling it with no arguments still reproduces the original baseline):

```
def build_model(n_features, activation="relu", learning_rate=0.01):
    model = keras.Sequential([
        layers.Input(shape=(n_features,)),
        layers.Dense(8, activation=activation,
                     kernel_regularizer=keras.regularizers.l2(1e-2)),
        layers.Dense(1),
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="mse", metrics=["mae"],
    )
    return model
```

No other layer, loss function, or regularization term was touched — the architecture (Input → Dense(8) → Dense(1)) is identical to the in-class version.

2.2 Sweep loop replacing the single training call

The single `model.fit(...)` call was wrapped in a nested `for activation in activations: for lr in learning_rates:` loop. Before each of the 9 fits, `np.random.seed(SEED)` and `tf.random.set_seed(SEED)` are re-issued so every run starts from the same nominal seed state (see the assumption on seeding in Section 3). For every run the notebook records, using `time.time()` around the fit call: the number of epochs actually run before EarlyStopping fired (`len(history.history['loss'])`), the wall-clock runtime, and train/validation MAE, RMSE, and R^2 (computed after inverse-transforming predictions

back to the original JRF units, exactly as in the in-class notebook). Results are collected into a pandas DataFrame, and each run's full loss history is kept in a dictionary keyed by (activation, learning_rate) so it can be plotted afterward.

2.3 New analysis and plotting cells

- A descriptive-statistics cell for the JRF column (mean, std, quartiles, coefficient of variation), as requested by the assignment.
- A retained single baseline run (relu, lr = 0.01) reproducing the in-class notebook's result, used as the point of reference for the sweep.
- A 3-panel MSE-vs-epoch plot (one panel per activation function, one line pair — train solid / validation dashed — per learning rate), which is the specific diagnostic plot the assignment asks for.
- Grouped bar charts of validation RMSE and epochs-to-convergence across all 9 combinations, and the same 4-panel actual-vs-predicted / loss-curve / scatter diagnostic from the in-class notebook re-run on the best-performing combination.

Nothing about the data loading, scaling, or train/validation split logic was modified.

3. Assumptions and Limiting Statements

- **Same 80/20 split and hidden-layer width for every run.** Task 3 asks only to vary the learning rule and activation function, so the split fraction (0.2) and hidden-layer width (8 neurons) were held fixed at the course defaults for all 9 combinations, isolating the two variables of interest.
- **Learning rate = “learning rule.”** The assignment's phrase “learning rule” is interpreted as the optimizer's learning-rate hyperparameter (the value explicitly called out as 0.01 in the course-default description), not a change of optimizer algorithm (Adam was kept throughout, consistent with the in-class notebook).
- **Seed reset is approximate, not exact.** `np.random.seed()` and `tf.random.set_seed()` are reissued before every run to make the 9 comparisons as fair as possible. In practice, TensorFlow/Keras does not guarantee bit-for-bit identical weight initialization when the global seed is reset mid-process versus set once at process start — this is why the sweep's `relu/lr = 0.01` result (59 epochs, validation RMSE 2.65 N) differs somewhat from the separately-run baseline cell using the same nominal hyperparameters (72 epochs, validation RMSE 2.17 N). Both numbers are reported honestly below rather than reconciled, and this run-to-run sensitivity is itself consistent with the in-class notebook's stated caveat that a network this small, trained on only ~80 examples, is noticeably sensitive to its random initialization.
- **EarlyStopping, not a fixed epoch budget, determines training length.** The reported “epochs” is the point at which validation loss stopped improving for 40 consecutive epochs (or the 500-epoch cap), not a fixed training duration purposely chosen per configuration. Runtime is therefore a joint measure of per-epoch cost (nearly constant across activations) and how many epochs the optimizer needed — the two are not fully separable from these results alone.
- **Runtimes are hardware/CPU-dependent.** Training ran on CPU only (no GPU available in this environment); absolute runtimes will differ on other machines, though the relative ordering across configurations (e.g., `lr = 0.001` taking far longer than `lr = 0.01/0.1`) should hold generally.
- **Small-N caveat.** With only 80 training rows and 20 validation rows, single-run validation R^2 is a noisy estimate; none of the 9 configurations should be read as a definitive ranking so much as a qualitative picture of how learning rate and activation choice affect convergence speed and stability on this dataset.

4. Results

4.1 Descriptive statistics for JRF ($n = 100$)

Statistic	Value (N)
Mean	132.87
Std. dev.	6.00
Min	119.68
25th pct.	128.21
Median	135.32
75th pct.	138.11
Max	142.73
Coefficient of variation	0.045

The measured JRF has a very low coefficient of variation (~4.5%) relative to the range of muscle PCSA inputs, meaning the target is a fairly tight, weakly-varying quantity around ~133 N. This is consistent with the difficulty

every MLP configuration below has in beating a near-constant prediction on the validation split (see negative R^2 values in most rows of Table 2) — with this little spread and only 80 training examples, a small MLP struggles to learn a signal much better than the sample mean, whereas the class's earlier linear/nonlinear static-optimization models used the known physical moment-arm relationships directly rather than inferring them empirically, and were not similarly starved for training examples.

4.2 Sweep results: epochs, RMSE, and runtime

Course-default baseline (relu, lr = 0.01, run once, outside the sweep loop): 72 epochs, 5.42 s, train RMSE 1.82 N, validation RMSE 2.17 N, validation R^2 = -0.175.

Activation	Learning rate	Epochs	Runtime (s)	RMSE train (N)	RMSE val (N)	R^2 val
tanh	0.1	43	3.47	2.24	1.92	0.078
relu	0.001	356	24.77	1.66	2.09	-0.085
tanh	0.001	192	14.10	1.91	2.12	-0.115
sigmoid	0.1	64	4.95	1.94	2.16	-0.162
sigmoid	0.01	232	16.74	1.96	2.17	-0.170
relu	0.1	86	6.35	1.86	2.20	-0.207
tanh	0.01	44	3.63	2.83	2.40	-0.430
relu	0.01 (sweep run)	59	4.42	1.81	2.65	-0.744
sigmoid	0.001	41	3.52	6.76	2.98	-1.211

Table sorted by validation RMSE (best first). The best-performing combination (tanh, lr = 0.1, highlighted) is the only one of the 9 with a positive validation R^2 .

4.3 How the outputs changed as learning rate and activation varied

Effect of learning rate (holding activation fixed):

- Epochs-to-convergence and runtime scale inversely with learning rate: lr = 0.001 took 192–356 epochs (14–25 s) to satisfy EarlyStopping, while lr = 0.01 and 0.1 typically converged in 40–90 epochs (3.5–6.5 s) regardless of activation — a ~5–8× difference in wall-clock cost for this architecture.
- The lowest learning rate (0.001) gave the smoothest, most monotonic loss curves (see Figure 1) but did not necessarily give the best validation error — for relu and tanh it was competitive, but it was actively harmful for sigmoid (RMSE_val = 2.98 N, the worst of all 9 runs) because the small gradient steps were not enough to escape sigmoid's initial low-gradient regime within the epoch budget before EarlyStopping fired at epoch 41.
- The mid learning rate (0.01, the course default) gave the worst validation RMSE for both relu and tanh in this sweep (2.65 N and 2.40 N respectively) — a reminder that the “default” setting is not automatically optimal.

Effect of activation function (holding learning rate fixed):

- Sigmoid was the most sensitive to learning rate of the three activations: its validation RMSE ranged from 2.16 N (lr = 0.1) to 2.98 N (lr = 0.001), and it also needed by far the most epochs at lr = 0.01 (232 epochs vs. 44–59 for the other activations) — consistent with the textbook vanishing-gradient tendency of sigmoid units away from their optimal learning rate.

- tanh at the highest learning rate (0.1) was the only configuration in the whole sweep to achieve positive validation R^2 (0.078), and its diagnostic plots (Figure 3) show it tracking the validation set's up-and-down pattern reasonably well, though the loss curves in Figure 3 also show validation loss beginning to climb after ~epoch 15 while training loss keeps falling — a mild but visible overfitting signature despite the L2 penalty and EarlyStopping.
- relu, the course default, was the most consistent activation across learning rates (validation RMSE 2.09–2.65 N) but never the single best performer in this sweep, and its very small learning rate run (0.001) needed the most total epochs (356) of any configuration.

Overall, learning rate had the larger effect on *how fast* training converged (epochs and runtime), while activation function had the larger effect on *how well* the network could fit the validation data once it converged — but neither factor moved the needle from “the network struggles to beat a near-constant prediction” (R^2 consistently near or below zero) to a clearly strong fit. That is consistent with the small-N caveat from the in-class notebook: with 80 training rows, hyperparameter choice matters less than simply not having enough data for an 8-neuron MLP to reliably out-perform the physically-grounded static-optimization models used elsewhere in the course.

5. Figures

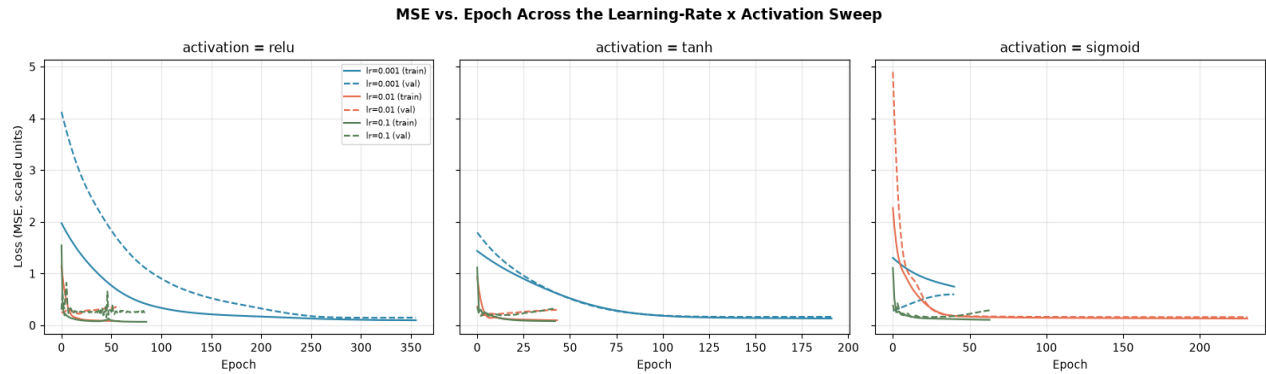


Figure 1. MSE vs. epoch for all 9 activation $\times$ learning-rate combinations (solid = training, dashed = validation; shared y-axis).

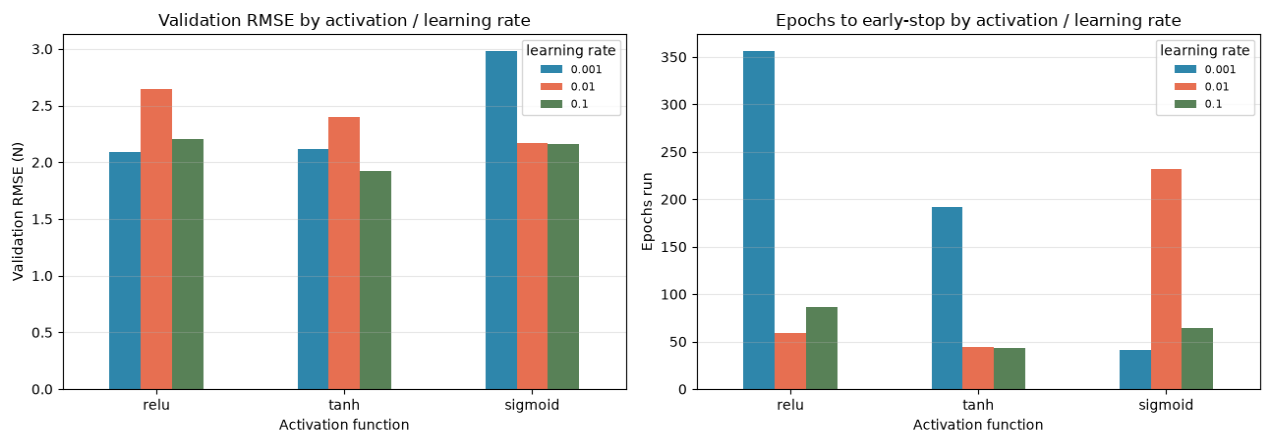


Figure 2. Validation RMSE (left) and epochs to early-stop (right), grouped by activation function and learning rate.

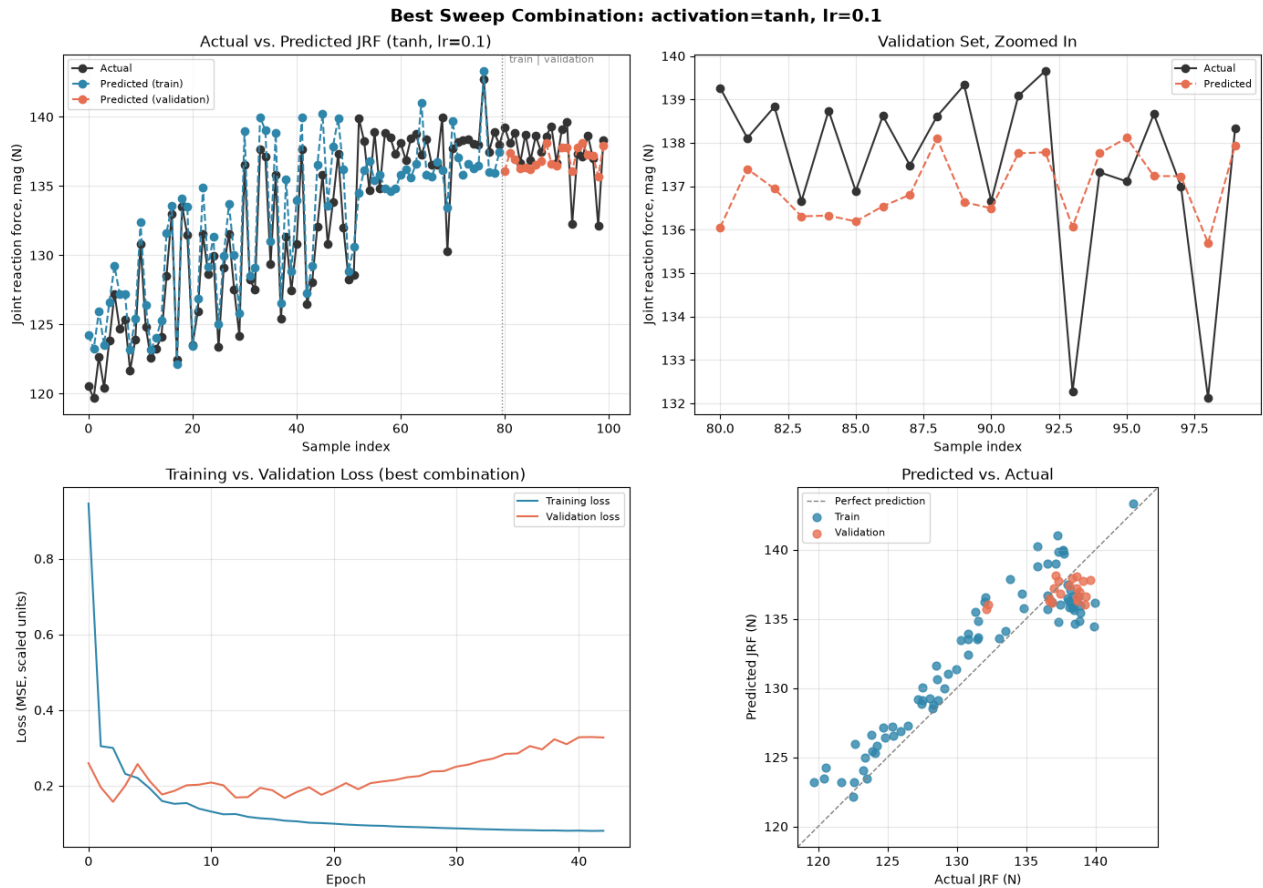


Figure 3. Four-panel diagnostic for the best sweep combination (tanh, learning rate = 0.1): actual-vs-predicted across the series, a zoomed validation view, the training/validation loss curve, and predicted-vs-actual scatter.

6. Conclusion

Turning the hidden-layer activation and Adam learning rate into parameters and sweeping a 3×3 grid (relu/tanh/sigmoid $\times$ 0.001/0.01/0.1) reproduced the course-default architecture as one point in that grid and showed that, on this 100-subject JRF dataset, learning rate mainly trades off training speed against a modest amount of accuracy, activation choice mainly trades off training stability, and the single best configuration found (tanh, lr = 0.1) still only reaches a weakly positive validation R^2 (0.078). This reinforces the in-class notebook's warning that a small MLP trained on ~ 80 examples is a noisy estimator relative to the deterministic, physically-derived static-optimization models used earlier in the course for the same JRF problem.